Common Concurrency Problems

Prepared by: Rene Andre B. Jocsing

Published: March 16, 2026 **Deadline:** March 26, 2026

This problem set explores one of the most practically important topics in concurrent systems: the bugs that real programs get wrong, and the strategies engineers use to prevent them. You will examine two broad classes of concurrency bugs identified through empirical study of real-world software, and analyze the landscape of techniques used to handle the most notorious of them — deadlock. This assignment requires careful reading of the provided chapter and substantial independent research to develop a thorough, well-supported analysis.

Background

Writing correct concurrent programs is notoriously difficult, even for expert engineers. Unlike sequential bugs, concurrency bugs are often non-deterministic: they may appear only under specific thread interleavings, making them hard to reproduce, diagnose, and fix. The consequences range from subtle data corruption to complete system deadlock.

A landmark empirical study by Lu et al. (2008) analyzed concurrency bugs in four major open-source applications — MySQL, Apache, Mozilla, and OpenOffice — and found that bugs cluster into recognizable patterns. The majority are *non-deadlock bugs*: atomicity violations, where a sequence of operations assumed to be atomic is not, and order violations, where the assumed ordering between two threads is never enforced. The remainder are *deadlock bugs*, arising from circular lock dependencies that leave threads permanently blocked.

Understanding these patterns matters because it shapes how we write, review, and reason about concurrent code. More importantly, each class of bug admits a distinct family of solutions — from simple lock insertion to sophisticated lock-free data structures built on hardware atomic instructions — each with its own trade-offs in correctness, performance, and complexity.

Learning Objectives

After completing this problem set, students should be able to:

- Explain the empirical basis for classifying concurrency bugs, referencing Lu et al. (2008)
- Distinguish atomicity violations from order violations, and identify the appropriate fix for each
- State, explain, and apply the four necessary conditions for deadlock (Coffman conditions)
- Describe and critically evaluate the four strategies for handling deadlock: prevention, avoidance, detection and recovery, and lock-free approaches
- Analyze real code scenarios and identify the class of concurrency bug present
- Compare trade-offs across deadlock-handling strategies with respect to concurrency, complexity, and generality
- Conduct independent research using academic sources well beyond the primary reading
- Synthesize technical concepts into a coherent, well-structured written argument

---

Essay Topic: Bugs, Deadlock, and the Cost of Concurrency

Using Chapter 32 of OSTEP (Arpaci-Dusseau & Arpaci-Dusseau, 2023) as your primary reference and additional academic sources, write a comprehensive essay that addresses the following four sections.

1. The Landscape of Concurrency Bugs

Introduce the empirical foundation of this chapter: the study by Lu et al. (2008) and its analysis of real concurrency bugs in production software. Your discussion should cover:

- The four applications studied and the overall distribution of bugs found — how many were non-deadlock versus deadlock bugs, and what does this distribution tell us about where programmers most commonly go wrong?
- Why concurrency bugs are particularly insidious compared to sequential bugs. What properties of concurrent execution — thread interleaving, non-determinism, the absence of enforced atomicity — make these bugs hard to find and reproduce?
- The significance of the finding that 97% of non-deadlock bugs are either atomicity or order violations. What does this concentration imply for the design of program analysis tools, code review practices, and programming language features?

2. Non-Deadlock Bugs: Atomicity and Order Violations

Describe the two major types of non-deadlock bugs in detail. For each type:

- Provide its formal definition as given by Lu et al. (2008)
- Walk through the concrete code-level example from the chapter, explaining precisely what assumption the code makes and how a specific thread interleaving violates that assumption
- Describe the fix, explaining *why* the synchronization primitive chosen (mutex, condition variable, etc.) restores the violated invariant

Then, reflect on the broader implications: if atomicity and order violations are so common and the fixes are often straightforward, why do these bugs persist in mature, widely-reviewed codebases? Consider the role of implicit assumptions, encapsulation, and the difficulty of reasoning about all possible interleavings.

3. Deadlock: Conditions and Prevention Strategies

Explain deadlock in depth. Begin by stating and explaining each of the four Coffman conditions, illustrating each with a concrete example in terms of threads and locks. Emphasize that all four conditions must hold simultaneously for a deadlock to occur — and therefore, breaking any one of them is sufficient to prevent it.

Then, describe the three prevention strategies the chapter presents:

- **Circular wait prevention** via total or partial lock ordering — how does enforcing a consistent acquisition order eliminate cycles? What are the practical challenges of maintaining such an ordering in a large codebase?
- **Hold-and-wait prevention** via atomic lock acquisition — how does acquiring all locks upfront prevent the condition? Discuss why this approach conflicts with encapsulation and reduces concurrency.

- **No-preemption prevention** via `pthread_mutex_trylock()` — explain the mechanism and introduce the concept of *livelock* as an emergent hazard of this approach. How does livelock differ from deadlock, and how can it be mitigated?

4. Beyond Prevention: Avoidance, Detection, and Lock-Free Approaches

Examine the remaining strategies for handling deadlock and reflect on the broader design space.

- **Deadlock avoidance via scheduling**: Explain the idea behind approaches like Dijkstra's Banker's Algorithm — using global knowledge of lock requirements to schedule threads safely. Why are these approaches only practical in limited environments? What real-world properties of general-purpose systems make them inapplicable?
- **Detection and recovery**: Describe how a deadlock detector using a resource graph can identify cycles and trigger recovery. What are the conditions under which this "let it happen and fix it" approach is pragmatically reasonable? What are its costs?
- **Lock-free approaches**: Explain how hardware atomic instructions such as compare-and-swap (CAS) can be used to build data structures that avoid locks entirely. Walk through the chapter's lock-free list insertion example and explain why CAS eliminates the possibility of deadlock. Discuss the limitations: why are lock-free structures harder to build correctly, and why do they not fully replace locks in general-purpose systems?

Finally, synthesize: given the full landscape of strategies — prevention, avoidance, detection, and lock-free approaches — argue which combination you believe is most appropriate for a general-purpose operating system kernel, and why. Acknowledge the trade-offs of the alternatives you did not choose.

Research Requirements

This essay demands substantial research well beyond Arpaci-Dusseau and Arpaci-Dusseau (2023) and Lu et al. (2008). Both are your starting point, not your finish line — essays that cite only these two will not meet the research standard for this assignment. You are expected to independently locate and engage with sources that deepen, contextualize, or challenge the ideas presented in the reading.

You must:

- Cite **at least three (3) academically sound sources** using **APA 7th Edition** format
- Arpaci-Dusseau and Arpaci-Dusseau (2023) counts as one source; Lu et al. (2008) counts as a second — you need **at least one additional source** that you found independently, beyond what the chapter already hands you
- Acceptable sources include:
  - Peer-reviewed papers from conferences (ACM SIGOPS, USENIX, etc.)
  - Technical books on operating systems, concurrency, or distributed systems (see: syllabus)
  - Official Linux kernel documentation on locking and synchronization
  - Academic journal articles on concurrency, deadlock, or synchronization primitives
- **Not acceptable**: Wikipedia, blogs, online forums, AI-generated content presented as original research

- Include a **References** section at the end following APA 7th Edition format
- The References section does **not** count toward your word count

The following are suggested entry points from the chapter's own reference list. They are freely available online and are directly relevant to the essay's content — but again, you are expected to go further:

- Coffman et al. (1971) — the original formalization of deadlock conditions
- Herlihy (1991, 1993) — foundational work on wait-free and lock-free synchronization
- Jula et al. (2008) — deadlock immunity and practical deadlock defense in real systems
- Harris (2001) — implementing non-blocking linked lists without locks

Strong essays will draw on sources the chapter does not mention. Use Google Scholar, IEEE Xplore, the ACM Digital Library, or your university library portal to find relevant work on topics such as: formal verification of concurrent programs, transactional memory as an alternative to explicit locking, real-world deadlock incidents in production systems, or the scheduling implications of lock-free data structures.

Format Requirements

- Write on **one-whole sheets of yellow pad paper only**
- Work together with your trio defined in this sheet to accomplish the essay
- Your essay must be approximately **2000 words** (acceptable range: 1800–2200 words)
- Essays outside this range will receive deductions
- **CRITICAL**: You must write the **cumulative word count at the left margin of each line**
  - Example: Line 1 ends at word 8, write `8` at the left margin
  - Line 2 ends at word 17, write `17` at the left margin
  - Continue throughout the entire essay
- Write **legibly in PRINT** (not cursive)
- Illegible answers will receive reduced credit
- **Minimal erasures**: If you make a mistake, neatly cross out with a **single line** (avoid correction fluid or tape!)
- Excessive corrections leading to illegible writing will result in deductions
- You may include hand-drawn diagrams, tables, or pseudocode to illustrate your points
- Such visual aids do not add to the essay's word count but can enhance clarity

---

Submission Instructions

1. Submission must occur **in-person during class hours** to the instructor on or before **March 26, 2026 (up to 5:30PM)**
2. All yellow pad sheets must be **stapled together** securely in the upper-left corner
3. Write your **FULL NAMES** on the top left of the first page
4. Write the **TOTAL WORD COUNT** on the top right of the first page
5. Include page numbers on the bottom right of each sheet

6. As per the syllabus, late submissions for lecture requirements will **NOT** be accepted except with university-sanctioned documentation (medical emergencies, family emergencies, etc.)

Academic Honesty

This is a **trio assessment** (groups of 3, or groups of 2 if the class size does not divide evenly). Discussion with your groupmates is expected and encouraged, but the final output must come entirely from your group. You may use AI tools (ChatGPT, Claude, Deepseek, etc.) for research and to aid your understanding of complex concepts, but you must read, comprehend, and synthesize the information yourselves.

The final essay must be written by hand in your own words. As per the syllabus, direct copying from AI outputs or other sources is a serious academic offense. Violations will result in:

- Automatic failure in the course (5.0)
- Referral to the university disciplinary committee
- Possible expulsion from the university

Proper citation of all sources is mandatory. Plagiarism (presenting others' work as your own) will be prosecuted to the full extent of university policy.

Evaluation Criteria

Your essay will be evaluated on five dimensions:

- **Technical Accuracy (30%)**: Correctness of OS concepts, bug descriptions, deadlock conditions, and mechanism explanations
- **Depth of Analysis (25%)**: Synthesis of concepts, engagement with trade-offs, critical thinking, and quality of the final argument
- **Research Quality (20%)**: Quality and integration of academic sources beyond the primary reading, proper APA citations
- **Organization & Clarity (15%)**: Logical flow, clear explanations, precise technical writing
- **Format Compliance (10%)**: Adherence to word count, line numbering, legibility, and submission requirements

See the detailed rubric on the final page for specific performance standards.

Tips for Success

- **Read the chapter thoroughly before writing**: This is your primary reference — understand every section before drafting
- **Go beyond the suggested sources**: The chapter's reference list is a floor, not a ceiling. Essays that only cite Arpaci-Dusseau and Arpaci-Dusseau (2023) and Lu et al. (2008) will score poorly on Research Quality
- **Divide the research, not just the writing**: Each member should independently explore sources for their section — then share findings before drafting begins
- **Plan before you write**: 2000 words across four sections is substantial — sketch an outline and agree on coverage before anyone picks up a pen
- **Use the code examples**: The chapter provides concrete bug scenarios; use them to ground your explanations, not just to summarize
- **Argue in Section 4**: The synthesis question asks for a position — pick a combination of strategies and defend it with trade-offs; don't describe all four equally
- **Write the essay together**: With three authors, it is tempting to write four sections independently and staple them. Resist this. Read and edit each other's sections so the essay reads as a unified piece
- **Count as you write**: Tracking cumulative word count per line from the start avoids painful recounting at the end

---

Common Pitfalls to Avoid

- **Confusing the two bug classes**: Atomicity and order violations are distinct — make sure your definitions and examples clearly distinguish them
- **Listing the Coffman conditions without explaining them**: Naming all four earns little credit; explaining each in terms of threads and locks, with examples, is what the prompt requires
- **Skipping livelock**: It is a required concept in Section 3 and a common omission
- **Treating Section 4 as a summary**: The synthesis question requires an argument, not a recap — make a claim and defend it
- **Citing only OSTEP and Lu et al.**: This is explicitly insufficient. The minimum is three sources, and Arpaci-Dusseau and Arpaci-Dusseau (2023) plus Lu et al. (2008) only gets you to two
- **Weak source integration**: Don't merely list references at the end — weave them into your argument where relevant
- **A disjointed essay**: Three authors can produce three disconnected pieces. Make sure transitions between sections are smooth and the essay has a coherent voice throughout
- **Format violations**: Forgetting line counts or exceeding word limits will cost you points regardless of content quality

---

Why Handwritten?

You may wonder why this assignment requires handwritten submission in an age of word processors and digital documents. This is a deliberate pedagogical decision designed to maximize your learning outcomes.

The AI Reality: Modern AI tools can generate technically accurate essays on operating systems concepts. However, this course aims to develop *your* understanding, not the AI's. The handwritten requirement creates a necessary friction point in the workflow.

Forced Engagement: If you use AI assistance for research or drafting, you cannot simply copy-paste the output. You must physically transcribe every word by hand. This process forces you to:

- **Read every sentence carefully**: You cannot transcribe text you haven't read
- **Process the information**: Handwriting is slower than typing, giving your brain time to process concepts as you write them
- **Identify gaps in understanding**: When you write something you don't understand, it becomes immediately apparent. You'll naturally pause, reread, and seek clarification
- **Engage with technical details**: Transcribing code snippets, conditions, and mechanism descriptions requires active attention to detail

Cognitive Benefits: Research in educational psychology consistently demonstrates that handwriting promotes deeper cognitive processing than typing. The act of forming letters, combined with the slower pace, enhances memory encoding and conceptual understanding. When you handwrite technical content, you're more likely to remember it during examinations and apply it in future coursework.

Academic Integrity: This format also serves an integrity function. While AI assistance is permitted for research and understanding, the final product must demonstrate *your* comprehension. A handwritten essay in your own words ensures the work is authentically yours. We can distinguish between genuine understanding and superficial reproduction.

Professional Preparation: In technical interviews and graduate school qualifying exams, you will often work through problems by hand on whiteboards or paper. This assignment provides practice in organizing complex technical arguments without digital aid — a valuable professional skill.

The goal is not to make your life difficult, but to ensure that the time you invest in this assignment translates into genuine learning. When you sit for future examinations, beyond the walls of the university, the concepts you've handwritten will be far more accessible in your memory than those you merely copy-pasted into a document.

References

See the problem set guide for the list of references used.

Grading Rubric

Criteria	Excellent (90–100%)	Good (75–89%)	Fair (60–74%)	Poor (0–59%)
Technical Accuracy (30%)	All OS concepts correct; mechanisms described precisely; calculations shown correctly; examples accurate; no conceptual errors.	Core concepts correct; minor inaccuracies in details; calculation errors or missing steps; examples mostly accurate.	Some correct concepts; significant errors in mechanisms; failed calculations; confused examples.	Fundamental misunderstanding; incorrect mechanisms throughout; no accurate calculations or examples.
Depth of Analysis (25%)	Synthesizes concepts across sections; explores tradeoffs deeply; concrete examples throughout; anticipates edge cases; critical thinking evident; goes beyond surface description.	Good analysis; explores some tradeoffs; provides adequate examples; shows solid understanding of implications and relationships.	Mostly descriptive; limited analysis; few examples; surface-level understanding; minimal exploration of tradeoffs.	Purely descriptive; no analysis; regurgitates definitions; fails to address prompt requirements; no critical thinking.
Research Quality (20%)	3+ high-quality academic sources; meaningful integration into argument; perfect APA citations; sources directly support claims; demonstrates independent research.	3 acceptable sources; generally correct APA format; relevant sources with adequate integration; some independent research evident.	Minimum sources met; citation errors; weak integration; sources tangentially relevant or poorly utilized.	Missing required citations; non-academic sources only; no APA format; plagiarism detected; no evidence of research.

Criteria	Excellent (90–100%)	Good (75–89%)	Fair (60–74%)	Poor (0–59%)
Organization & Clarity (15%)	Logical flow between sections; clear topic sentences; smooth transitions; precise technical writing; arguments build coherently; easy to follow throughout.	Generally organized; mostly clear explanations; adequate transitions; readable throughout with minor structural issues.	Weak organization; unclear explanations; choppy transitions; difficult to follow in places; some technical writing issues.	Incoherent structure; incomprehensible explanations; no clear argument thread; impossible to follow.
Format Compliance (10%)	Word count within range; cumulative line counts correct throughout; legible print handwriting; minimal/no erasures; proper References section; all requirements met.	Minor word count deviation (<50 words); line counts mostly correct with few gaps; mostly legible; some erasures but readable; References present with minor errors.	Significant word count deviation (50–100 words); many missing line counts; partially illegible; excessive erasures; incomplete References.	Word count requirements ignored; missing line counts; illegible handwriting; violates format requirements; no References; critical submission violations.